Lab 4 (*Optional*): Automated Deployment (CD) with PaaS (Render)

I. Learning Objectives

After completing this lab, you will be able to:

- Deploy a containerized application to the cloud (Render.com)
- Setup a hosted PostgreSQL database
- Configure GitHub Actions to automatically trigger a deployment when code is merge to `main`.

II. Prerequisites

- **Lab 3 Completed:** Your project's MUST succesfully build and push a Docker Image to Docker Hub (eg., `YourUsername/devops-group-1:latest)`.
- **Render account:** Sign up for a free account at render.com (You can login with GitHub)
- **New Branch:** Start fresh

```
$ git checkout main
$ git pull origin main
$ git checkout -b feature/add-render-cd
```

III. Conceptual Check: Local vs. Production

You might be wondering: *"We wrote a `docker-compose.yml` file in Lab 2. Why aren't we using `docker-compose up` on Render?"*

This is a fundamental difference between development and cloud deployment (PaaS):

Feature	Local (Development)	Production (Render Cloud)
How it runs	Run `docker-compose up` on your laptop.	Render runs your **Docker Image** automatically. No manual commands.
Database	A temporary container defined in `docker-compose.yml`.	A real, persistent **Managed Database** provided by Render.
Networking	App connects to `db:5432` (internal network).	App connects to a secure URL (e.g., `dpg-xyz.oregon-postgres...`).

Key Takeaway: You **still need** `docker-compose.yml` so your team can run the app locally. But on Render, we deploy the App and Database as two separate, connected services.

IV. Part A: Create Database on Render

First, we need a live database for our application to connect to.

Step A.1: Create PostgreSQL Service

1. Log in to your **Render Dashboard**.
2. Click "**New +**" button -> Select "**PostgreSQL**".
3. **Name:** `db-group-1` (Unique name).
4. **Region:** Singapore (nearest to Vietnam)
5. **PostgreSQL Version:** 16 (or latest).

6. **Instance Type:** Select "**Free**".
7. Click "**Create Database**".

Step A.2: Get Connection Details

Wait a few minutes for the database to be "Available".
Then find the "**Connections**" section.

- Copy the "**Internal Database URL**" (Save this for later).
- Copy the "**External Database URL**" (Save this too).

> **Note:** The "*Free*" tier database expires after 30 days, which is enough for this project.

V. Part B: Deploy the App on Render

Now we will tell Render to pull your Docker Image from Docker Hub and run it.

Step B.1: Create Web Service

1. Click "**New +**" -> Select "**Web Service**".
2. Select "**Deploy an existing image from a registry**".
3. Click "**Next**".
4. **Image URL:** Enter your Docker Hub image path (from Lab 3).
   Example: `YourUsername/devops-group-1:latest`
5. Click "**Next**"

Step B.2: Configure Service

- **Name:** app-group-1.
- **Region:** Same as your Database (Singapore).
- **Instance Type:** Select "**Free**".
- **Environment Variables:** (This connects App to DB). Click "*Add Environment Variable*".

- Key: DB_CONNECTION_STRING (Or separate vars if your app requires it)
- Value: Paste the **Internal Database URL** from Step A.2.
- *Note: If your code expects `DB_HOST`, `DB_USER` separately, you must add them one by one using the info from the connection string.*

Click "**Create Web Service**".

Step B.3: Verify Deployment

Render will now pull your image and start it. Watch the logs on the screen.

- When you see "*Your service is live*", click the URL (e.g., `https://app-group-x.onrender.com`).
- You should see your app running!

VI. Part C: Automate Deployment (The “CD” Part)

Currently, Render deploys the image we manually specified.
We want Render to **auto-deploy** whenever we update the image in GitHub Actions.

Step C.1: Get Deploy Hook

1. Go to your **Web Service** (App) settings on Render.
2. Scroll down to "**Deploy Hook**".
3. Click "**Create Deploy Hook**" (if not visible).
4. Copy the URL. It looks like: `https://api.render.com/deploy/srv-c.....`

This URL is a "*magic button*". Whenever someone visits this URL, Render re-downloads the latest image and restarts the app.

Step C.2: Add Secret to GitHub

1. Go to your Group Repo on **GitHub** ⇨ **Settings** ⇨ **Secrets and variables** ⇨ **Actions**.

2. Add a New Repository Secret:

    - Name: `RENDER_DEPLOY_HOOK`

    - Value: Paste the URL you just copied.

Step C.3: Add Secret to GitHub

Open your `.github/workflows/ci.yml`, file in VS Code. Add `deploy` job at the end.

```
jobs:
  build-and-test:
    # ... (Your existing job stays the same) ...

  deploy:
    name: Deploy to Render
    runs-on: ubuntu-latest
    needs: build-and-test # Wait for tests & build to finish
    if: github.ref == 'refs/heads/main' # Only deploy on
main branch
    steps:
      - name: Trigger Render Deployment
        run: curl -X POST ${{
secrets.RENDER_DEPLOY_HOOK }}
```

> **Explanation:** This job simply uses `curl` (a tool to make web requests) to hit the "magic button" URL. This tells Render: "Hey, I just pushed a new image to Docker Hub. Please update!".

VII. Part D: The Final Test

Step D.1: Commit and Push

```
$ git add .github/workflows/ci.yml
$ git commit -m "ci: Add Render deployment hook"
$ git push origin feature/add-render-cd
```

Go to GitHub and open a Pull Request for your branch.

Step D.2: Merge Pull Request

1. Open a PR on GitHub.
2. Check that the `deploy` job is **SKIPPED** (Correct, because it's not main).
3. **Merge** the PR.

Step D.3: Watch the Magic

1. Go to the "**Actions**" tab. Watch the workflow running on main.
2. The `build-and-test` job runs (Building new image ⇨ Pushing to Docker Hub).
3. The `deploy` job runs (Triggering Render).
4. Go to your Render Dashboard. You should see a new deployment starting automatically!

Step D.4: Verify Visual Change

To be 100% sure:

1. Make a visible change in your code (e.g., change the title in **index.html** to "**My DevOps Group Project**").
2. Commit, Push, PR, Merge.
3. Wait for the pipeline to finish.
4. Refresh your live Render URL. You should see the text change.